USD8 — Security Audit Fix Report

Responds to the BONISTECH security audit report (bonis.tech) of commit **6efab51299d551ed45b20d82b305d300e9f01d7b**. All five findings are addressed in commit **460961d** on `github.com/Usd8-fi/usd8-core` (main). Each fix ships with a regression test that was confirmed failing on the audited tree and passing after the change. Full suite after fixes: 536 passed, 0 failed, 5 skipped.

Summary

ID	Title	Status	Files
M-01	Admin claimant can steal other claimants' insured tokens	Fixed	SharedBase.sol, DefiInsurance.sol
L-01	Funded rewards become unclaimable after share inflation	Fixed	SingleAssetCoverPool.sol
L-02	Pool exits fail after share supply reaches its limit	Fixed	SingleAssetCoverPool.sol
L-03	Treasury reserve can be impaired by vault share rounding	Fixed	Treasury.sol
L-04	Incident claims can lose their payout route	Fixed	Registry.sol

M-01 — Admin claimant can steal other claimants' insured tokens

Root cause. `cancelClaim` and `_resolveWithoutSettlement` reduced `escrowedInsuredTokens` before returning boosters. The ERC-1155 `onERC1155Received` hook let a contract claimant with admin rights re-enter `sweepToken`, which was unguarded and computed its surplus from the already-reduced liability.

Fix. (1) `SharedBase` now inherits `ReentrancyGuardTransient`; `sweepToken` and `sweepETH` are `nonReentrant`, closing the entry point for every contract that inherits `SharedBase`. (2) In `cancelClaim`, `_resolveWithoutSettlement` and `_resolveClaimBond` the escrow accounting is released only after the corresponding transfer, so a hook fired mid-operation can only observe an over-reserved balance, which is never sweepable.

Test. `test/SweepReentrancy.t.sol`: a contract claimant re-enters `sweepToken` from the booster hook; the sweep reverts with `ReentrancyGuardReentrantCall`, other claimants' escrow is intact, and a legitimate surplus sweep still works.

L-01 — Funded rewards become unclaimable after share inflation

Root cause. Each checkpoint computed `elapsed × rate × REWARD_SCALE / earningShares` and discarded the remainder. With sufficiently inflated share supply every hourly slice floored to zero while `lastUpdateTime` still advanced, stranding the stream.

Fix. A new `rewardResidual` storage variable (appended last; beacon upgrade layout-safe) carries the division remainder into the next checkpoint. `_checkpointGlobalRewards` adds the residual to the numerator, credits the quotient and stores the new remainder; `_rewardPerShare()` includes it so `earned()` previews match. No scaled unit is ever lost; the residual is bounded by `earningShares − 1`. Residual accumulated under one share count is credited at the next, an accepted dust-level trade-off.

Test. `test/RewardResidual.t.sol`: the auditor's scenario (7-day stream, 168 hourly zero-value self-transfers by a shareless account) earns 0 on the audited tree and the full stream after the fix; fragmented and single checkpoints credit identical totals; conservation holds across a share-count change.

L-02 — Pool exits fail after share supply reaches its limit

Root cause. `maxMint` did not reserve headroom for the `totalSupply + 10**_decimalsOffset()` term used by the inherited ERC-4626 conversions, so supply could be pushed to a range where the partial-supply exit conversion overflowed.

Fix. `_shareHeadroom()` returns `type(uint256).max − 10**_decimalsOffset() − totalSupply()`. `maxMint` and `maxDeposit` are capped by it (composed with the existing `depositCap`), and `_deposit` enforces it as a last line with `ShareSupplyHeadroomExceeded`. At the limit the pool stops accepting new shares while exits, rewards and claims remain fully functional. The pool may stop accepting deposits in that state, which is the intended failure mode.

Test. `test/ShareHeadroom.t.sol`: supply is filled to the exact limit; the next share is refused via both `mint` and `deposit`; the auditor's partial-supply exit (one share retained, rest queued) settles and pays; headroom and deposit cap compose correctly.

L-03 — Treasury reserve can be impaired by vault share rounding

Root cause. The ERC-4626 adapter's tolerance of one share plus one unit scales with vault share price, so a donation-inflated vault could absorb a material fraction of a deployment while the Treasury accepted the resulting reserve reduction.

Fix. `Treasury.depositToStrategy` snapshots `getReserveBalance()` before and after `strategy.deploy()` and reverts with `StrategyDeployReserveLoss(before, after)` if the aggregate USDC-denominated reserve falls by more than the existing `RESERVE_CHECK_TOLERANCE` (100 base units, the same allowance mint and redeem already use). The check lives at the Treasury boundary so it covers every strategy adapter.

Test. `test/StrategyDeployReserve.t.sol`: the auditor's 1-unit seed + 500 USDC donation vault reduces the reserve from 1000 to 833.33 USDC on the audited tree; after the fix the deployment reverts and the reserve is unchanged. A healthy vault with ordinary rounding still deploys.

L-04 — Incident claims can lose their payout route

Root cause. `setDefiInsurance(address(0))` intentionally bypassed the active-incident check so a stuck module could be cleared, but the timelock operation carried no reference to the module or incident it was scheduled for. A ready operation left unexecuted could be run by anyone during a later incident, unregistering a healthy module and voiding that incident's payouts.

Fix. The two responsibilities are split. `setDefiInsurance` is now set-only (non-zero, no active incident on either side). The emergency brake is a new `clearDefiInsurance(expectedModule, expectedIncidentId)`: it reverts with `ModuleChanged` if the registered module differs, and with `IncidentChanged` if the module reports a live incident other than the one bound into the operation. A module whose `activeIncidentId()` reverts is still clearable via `try/catch`, preserving the original escape hatch. Incident ids are monotonic, so no additional storage is needed. Deployment scripts only set the module and are unaffected.

Test. `test/SingleAssetCoverPool.t.sol`: an operation bound to incident 1 executed while incident 2 is live reverts and the module stays registered; the brake bound to the live incident still works mid-incident; module mismatch, no-incident, reverting-module and zero-address cases are covered.

Verification

For every finding the new test was run against a throwaway worktree of the audited tree and observed to fail with the exploit succeeding (M-01: escrow swept; L-01: 0 rewards earned; L-02: overflow path; L-03: 166.67 USDC reserve loss; L-04: n/a, new API), then against the fixed tree and observed to pass. `forge test` on commit 460961d: 536 passed, 0 failed, 5 skipped.